

AXON-RFC-006 Appendix B (v0.6): Pages as the First Reference Realisation of the Resource Execution Model

HTTP as an invocation adapter, not a new protocol surface
Three normative invariants: Adapter Purity, Resource Identity,
Deterministic Projection

Silan Hu
National University of Singapore

Draft v0.6, May 4, 2026

Abstract

This appendix specifies the canonical way an agent-native system maps a legacy world (HTTP) into the invocation graph of EasyNet, using the act of publishing-a-folder-as-a-website as the smallest reference instance. The construction is not a feature. It is a paradigm: HTTP is a transport adapter for ability invocations whose subjects are resources; the bytes a browser receives are deterministic projections of resource state, not opaque payloads of a private protocol. The appendix establishes three normative invariants — Adapter Purity, Resource Identity, Deterministic Projection — under which **publish**, **fetch**, and **unpublish** form the create / read / delete face of a Resource Execution Model that subsequent reference systems (file storage, conversation streams, mission outputs) inherit without re-derivation. The implementation content from v0.5.1 is retained as §6 to witness that the model can be realised on current EasyNet-Cli + EasyNet infrastructure, but the load-bearing claim of v0.6 is semantic, not engineering. The MVP — one folder, two files (**hello-world.html**, **style.css**), one browser fetch, one kernel-enforced sandbox — is the existence proof.

Contents

1	What changed from v0.5.1	1
2	The paradigm	3
2.1	Statement	3
2.2	The Resource Execution Model	3
2.3	The minimum reference instance: pages	3
3	Assumptions	4
4	The three normative invariants	4
4.1	INV-1 — Adapter Purity	5
4.2	INV-2 — Resource Identity	5
4.3	INV-3 — Deterministic Projection	6
4.4	Why these three, no fewer, no more	6
5	State transition model in detail	7
5.1	publish: create(resource)	7
5.2	fetch: read(resource)	7
5.3	unpublish: delete(resource)	8
5.4	Read sandbox: a semantic obligation, not an implementation detail	8

6	Receipt semantics	8
6.1	Two tiers, derived from the Model	8
6.2	What receipts witness, in this reference system	9
6.3	Linkage to URA v2 plan	9
7	URL = invocation, in this reference system	9
7.1	The transformation	9
7.2	Why this is not just a routing rule	10
8	Reference implementation (informative)	10
8.1	Inherited from v0.5.1	10
8.2	The MVP: a witness, not a deliverable	10
8.3	Test stratification	10
9	Beyond the reference system	11
9.1	What this RFC unlocks (without specifying)	11
9.2	What this RFC deliberately leaves to future work	11
10	Coverage check	12

1 What changed from v0.5.1

v0.5.1 was the last feature-design draft. v0.6 is a paradigm rewrite. The headline change:

v0.5.1	v0.6
Stated as a feature (“how to publish a folder as a website”)	Stated as a reference system (“how legacy HTTP enters the invocation graph”)
Most paragraphs described how current code implements the feature	Every paragraph in the main body answers the question <i>what semantic constraint must this system satisfy?</i> ; implementation moves to §6
Five engineering invariants (PG-INV-1 .. PG-INV-5) were ad-hoc, scoped to “pages”	Three archi-

v0.5.1’s content has not been deleted. Its implementation specifics are preserved verbatim in §6 (reference implementation appendix). The change is in load-bearing-ness: v0.6’s main body is what survives if every implementation choice in §6 is replaced.

2 The paradigm

2.1 Statement

[normative] HTTP is a transport adapter for agent-native invocations. A browser GET on a published page is not a new protocol surface; it is a transport rendering of the invocation tuple `invoke(caller, callee, ability, subject, nonce, causal_context, args)` whose *ability* is a project-owned read primitive, whose *subject* is the project’s resource identity, and whose returned payload is a deterministic projection of that resource’s state.

2.2 The Resource Execution Model

[derived] The minimal mechanism the paradigm requires:

State	:	Resource
Transition	:	Ability
Invocation	:	f(resource, args)
Receipt	:	proof of transition

publish	=	create(resource)	canonical, state-changing
fetch	=	read(resource)	operational, state-preserving
unpublish	=	delete(resource)	canonical, state-changing

A reference system is a concrete instantiation of this model. This appendix specifies the **pages** reference system; the companion appendix A specifies the **page** reference system (single document); future RFC-006-C .. F will instantiate further reference systems (file storage, conversation streams, mission outputs, agent-built API surfaces) on the same Model.

Note

The Model is intentionally narrower than “everything that has URA”. Not every resource is a Resource-Execution-Model citizen; only those for which a system declares **create/read/delete** primitives over the resource and binds them to ability URIs. v0.6 establishes Pages as the first such instantiation; v0.7 will lift the Model to a standalone specification that this appendix references.

2.3 The minimum reference instance: pages

[derived] Among possible Resource-Execution-Model instantiations, “publishing a folder of static bytes for HTTP fetch” is the smallest one with all four ingredients: a resource (the project), a state (the published bytes), three clear transitions (publish/fetch/unpublish), and a legacy transport (HTTP) that must enter the invocation graph through the adapter.

It is therefore the canonical first instance. Anything more ambitious adds payload, not paradigm. Anything less ambitious omits a transition class (e.g., a system without **delete** cannot witness the resource’s full lifecycle).

3 Assumptions

[normative] v0.6 is written under the following assumptions. Each is stated explicitly so that downstream implementation work (URA v2 plan, admission upgrades, receipt persistence) can be discharged independently while this appendix’s semantic claims remain stable.

Assumption — [

A-1: Resource identity exists] There exists, in EasyNet, a notion of resource identity `resource/<id-part>/<path-part>` satisfying URA v4.1.5 §A.URA-7 (three URA segments). The identity is consumable by admission, by receipts, and by the ability registry. Whether the identity is content-addressed or user-defined is an INV-2 question; whether it is enforced today by ent schema or deferred to URA v2 plan §Phase 2 is an implementation question and does not affect this appendix.

Assumption — [

A-2: Admission enforces `subject_kinds`] For each ability registered in EasyNet, the ability contract declares a `subject_kinds` whitelist; admission rejects invocations whose subject URA kind is not in the whitelist. Whether the enforcement is in code today or pending URA v2 plan §Phase 5 does not affect this appendix.

Assumption — [

A-3: Receipts can be chained] The receipt store can record a receipt’s `prev_receipt_hash` and walk the chain backwards from a leaf receipt. Whether chain-walking is in production today or pending URA v2 plan §Phase 6 does not affect this appendix; INV-3’s verification machinery uses chain-walking, but a non-walking store still satisfies the read-side invariants v0.6 places on individual receipts.

Assumption — [

A-4: Ability ownership widens to {user, agent, hub, resource}] The ontology theorem in `ONTOLOGY_AGENT_ABILITY.md` §3.2 states “Ability belongs to exactly one Agent.” v0.6 assumes the equivalent post-amendment statement: “Ability belongs to exactly one of {user, agent, hub, resource}.” The Phase B amendment (this plan) discharges this assumption. Until it lands, v0.6 reads the wider claim into v4.1.5 ability URI grammar (whose tail is already `<owner-id>.<name>`, with `<owner-id>` typed as “the URA kind that names the owner”).

4 The three normative invariants

4.1 INV-1 — Adapter Purity

Invariant — INV-1: Adapter Purity

ability/01HUB.pages.serve, the ability that bridges HTTP into the invocation graph for the pages reference system, **MUST** be a pure transport adapter. Specifically:

- It **MUST NOT** mutate any resource state.
- It **MUST NOT** emit canonical receipts.
- It **MAY** emit operational receipts (lossy observability) recording the HTTP request that triggered the forwarded invocation.
- It **MUST** delegate the read entirely to the project-owned ability `<user>.<project>.page.fetch` via the standard `forward_invoke` dispatch path.
- It **MUST NOT** cache or transform the bytes returned by `page.fetch`, beyond mechanical HTTP framing (Content-Type, Content-Length, ETag derivation from byte hash).

[derived] Adapter Purity is the invariant that prevents the HTTP transport from accidentally becoming a parallel state plane. Without it, a Hub implementation could (a) mutate project state during a serve, (b) record state-changing canonical receipts that did not originate from a project’s own ability, or (c) introduce side-channels through which an operator could publish bytes without ever invoking `publish`. Adapter Purity rules these out at the specification layer, before they can arise at the implementation layer.

4.2 INV-2 — Resource Identity

Invariant — INV-2: Resource Identity

Every published project **MUST** correspond to a resource identity expressed as a v4.1.5-conformant URA. The identity has two valid forms:

- **v0 transitional form (this appendix).** The identity is user-defined: `resource/<user>.<project>`. Uniqueness is enforced per user at publish time. This form is acceptable for single-host serve where federation is not yet exercising the identity.
- **v1 mandatory form (next appendix revision).** The identity **MUST** be content-addressed: the `<project>` component is the prefix of a Merkle hash over the canonicalised folder content. Aliases (mutable names pointing at content hashes) are first-class, the way Git refs name commits.

v0 publishes **MUST** record their transitional status in the publish receipt’s `state_transition` payload so that a future verifier can distinguish a v0 publish from a v1 one.

[derived] INV-2 separates “the resource exists” from “the resource has a stable, verifiable identity.” v0 publishes are useful for development and single-host operation but are not sufficient for federation, audit, or third-party caching. v1’s content-addressed form is what makes the resource a citizen of the network rather than a folder on Alice’s laptop.

Boundary

INV-2 explicitly declares that v0 is transitional. A v0 system without a clear v1 migration path would not be conformant. The URA v2 plan §Phase 6 (content-addressed blob store) provides the substrate for v1's identity computation; the migration is mechanical once that substrate exists.

4.3 INV-3 — Deterministic Projection**Invariant — INV-3: Deterministic Projection**

`ability/<user>.<project>.page.fetch` **MUST** be a deterministic projection of the project's resource state. For a given resource identity R and request path p :

$$\text{fetch}(R, p) = \text{bytes}(R, p)$$

where $\text{bytes}(R, p)$ is fully determined by R and p and contains no host-, time-, or environment-dependent components.

It follows that:

- Two hosts that both claim resource identity R **MUST** return byte-identical responses for the same p .
- A receipt for `fetch(R, p)` **MUST** bind R , p , and `sha256(bytes)` so a verifier auditing the chain can confirm the projection invariant after the fact.
- `fetch` **MUST NOT** introduce dynamic content (timestamps, IP-derived data, randomised banners). Such content belongs to a different ability whose contract explicitly admits non-determinism.

[derived] INV-3 is the invariant that makes the Resource Execution Model federate. Without it, two hosts claiming the same resource identity may return different bytes; with it, a caller (or a third-party CDN) can rely on identity alone for content equivalence. It is also the invariant that makes content-addressed resource identity (INV-2 v1) verifiable: the hash of the projection equals the hash that names the resource.

Note

INV-3 does not forbid all dynamic web content; it forbids dynamic content from `page.fetch`. A reference system that needs dynamic content (e.g., personalised dashboards) must introduce a separate ability whose contract declares non-deterministic projection, and the bytes from that ability flow through a different HTTP route (not under `*.*.pages.<realm>`) so callers know to expect non-determinism.

4.4 Why these three, no fewer, no more

[derived] Following the degeneracy method of AXIOM.tex §why-six:

- **Without INV-1**, the HTTP boundary becomes a second authoritative state plane. Any operator with access to the Hub's process can rewrite project state without invoking `publish`. The receipt chain is no longer the single source of truth.
- **Without INV-2**, two systems can claim the same name for different bytes. Federation collapses; third-party caching is unsafe; receipts cannot be verified.
- **Without INV-3**, identical names on two hosts return different bytes. Content cannot be cached without consulting an authoritative origin; the Hub is forced into a pull-through

cache role, which contradicts INV-1's purity.

Each invariant blocks a class of failure that the others do not address; removing any one collapses the model.

5 State transition model in detail

5.1 publish: create(resource)

[normative] `publish` is the canonical transition that creates a project resource and registers its `page.fetch` ability.

```

caller    = caller_uri           (typically the publishing user
                                   or one of their agents)
callee    = callee_uri          (the user URA hosting the project)
ability    = ability/<user>.pages.publish
subject    = user/<user>         (creating-self transition;
                                   the project resource does not
                                   yet exist)

args        = {
  folder:      absolute path on owner host,
  project_id:  URA-safe segment (v0 transitional, v1 derived),
  visibility:  "public" | "private" | "scoped"
}

```

Pre-state: no resource with identity `resource/<user>.<project>` exists.

Post-state: a resource with that identity exists; an ability `ability/<user>.<project>.page.fetch` is registered; the resource's bytes are (semantically) the content of the published folder.

Receipt class: canonical (state-changing). The receipt's `state_transition` payload **MUST** carry:

- `post_resource_uri`: the minted resource URA
- `identity_form`: "v0_user_defined" or "v1_content_hash" (per INV-2)
- `visibility`
- if v1: `content_hash` (full sha256, not just prefix)

5.2 fetch: read(resource)

[normative] `fetch` is the operational transition that reads bytes from the project resource.

```

caller    = caller_uri           (typically hub/01HUB during
                                   HTTP-adapted invocation)
callee    = callee_uri          (host agent of the project)
ability    = ability/<user>.<project>.page.fetch
subject    = resource/<user>.<project>/<path>
args        = { path: <path> }

```

Pre-state: resource exists; `<path>` resolves to a regular file inside the published root (per the read sandbox below).

Post-state: identical to pre-state. `fetch` mutates nothing.

Receipt class: operational (state-preserving). Per INV-3, the receipt **MUST** bind `<path>` and `sha256(bytes)`; the byte hash makes the projection invariant verifiable from the receipt alone.

5.3 unpublish: delete(resource)

[normative] unpublish is the canonical transition that revokes the resource.

```

caller    = caller_uri
callee    = callee_uri                      (host agent of the project)
ability    = ability/<user>.pages.unpublish
subject    = resource/<user>.<project>
args      = {}

```

Pre-state: resource with the given identity exists.

Post-state: resource no longer exists; the project’s `page.fetch` ability is unregistered; subsequent `fetch` attempts fail with the standard “no such ability” admission rejection.

Receipt class: canonical. The receipt’s `state_transition` payload binds the pre-state `content_hash` (when known) so the deletion is itself auditable: a verifier walking the chain knows what bytes existed at the moment of revocation.

5.4 Read sandbox: a semantic obligation, not an implementation detail

[normative] The read sandbox is the mechanism by which INV-3 becomes enforceable in practice. `page.fetch`’s callee **MUST** restrict path resolution to the published folder root using a kernel-enforced contract:

- Path resolution **MUST NOT** escape the folder root, regardless of `..` components, symbolic links, or filesystem mutations between calls.
- The contract **MUST** be at the kernel layer (e.g., `openat2(2)` with `RESOLVE_BENEATH | RESOLVE_NO_SYMLINKS` on Linux), not at the user-space layer (`realpath()+prefix-check`), because user-space checks race the filesystem (TOCTOU).

The sandbox is not a security feature added on top of the Model; it is the mechanism by which the Model’s deterministic projection (INV-3) holds against adversarial inputs. A path that the kernel refuses to resolve cannot be a member of the projection’s domain.

6 Receipt semantics

6.1 Two tiers, derived from the Model

[normative] The transitions in §5 sort naturally into two receipt classes:

Tier	For transitions that are	Used by
Canonical	state-changing (publish, unpublish)	<code>prev_receipt_hash</code> chain; persistent; walkable
Operational	state-preserving (fetch, serve)	lossy ring buffer; observability; aggregable rates

The classification is not a heuristic; it is a direct consequence of the Resource Execution Model. Canonical transitions advance state and therefore must be durably recorded for audit. Operational transitions read state without advancing it and would saturate the canonical chain under

any realistic load (an SPA visit yields tens of fetches per session); their receipts are observability artefacts, not state-history artefacts.

6.2 What receipts witness, in this reference system

[normative] For each transition class:

- **publish receipt** witnesses “user U minted resource R with content H at time t , visibility V ”.
- **unpublish receipt** witnesses “user U revoked resource R (last known content H) at time t' ”.
- **fetch receipt** witnesses “a request for path p on resource R returned bytes whose hash is h_p ”. By INV-3, two hosts that both claim R must produce the same h_p for the same p — a verifier comparing two operational receipts can detect deterministic-projection violation.
- **serve receipt (optional)** witnesses “the Hub received an HTTP request for R/p at time t and forwarded it to callee C ”. By INV-1, this receipt carries no state-change information; it is purely observability.

6.3 Linkage to URA v2 plan

[derived] The receipt-store substrate the URA v2 plan §Phase 6 introduces (chain walking, indexed by subject URA, indexed by ability URA) is exactly the substrate this RFC’s receipts inhabit. v0.6 makes no demands on receipt encoding beyond what URA v2 plan delivers; conformance to v0.6 requires only that the canonical/operational distinction is honoured.

7 URL = invocation, in this reference system

7.1 The transformation

[normative] The Hub binds a wildcard host pattern, `*.*.pages.<realm>`, on its TLS listener. An HTTP request:

```
GET /<path> HTTP/1.1
Host: <project>.<user>.pages.<realm>
```

is translated, by `ability/01HUB.pages.serve`, into:

```
caller    = hub/01HUB
callee   = callee_uri (host of <user>.<project>)
ability   = ability/<user>.<project>.page.fetch
subject   = resource/<user>.<project>/<path>
args      = { path: "<path>" }
```

The transformation is mechanical:

- Subdomain `<project>.<user>` → URA id-part `<user>.<project>` (URL leads with project for human readability; URA leads with user for owner-first identity).
- URL path → URA path-part verbatim.
- HTTP method `GET` → `page.fetch` (the only verb in this reference system; non-GET methods are rejected at the Hub).

7.2 Why this is not just a routing rule

[derived] By INV-1, the Hub’s serve ability does not own the read; it only forwards. By INV-3, the bytes the Hub returns are the same bytes any other client of `page.fetch` receives. Together, INV-1 + INV-3 mean that the URL form is a projection of the URA form: every URL has a URA, every URL’s response equals the URA’s invocation result, and the correspondence is verifiable through the receipt chain.

This is the load-bearing claim of the paradigm. HTTP is no longer “the API”; it is a viewing surface onto the invocation graph.

8 Reference implementation (informative)

This section is non-normative. It witnesses that the v0.6 specification can be realised on current EasyNet-Cli + EasyNet infrastructure, using the addressing form, sandbox mechanism, and receipt classification preserved from v0.5.1. Conforming implementations are not required to follow these specific choices.

8.1 Inherited from v0.5.1

The v0.5.1 implementation surface is preserved in this section: ability namespace formation (`<user>.<project>.page.fetch`), URL form (`<project>.<user>.pages.<realm>/<path>`), the `openat2(2) + RESOLVE_BENEATH` sandbox, the allow-list MIME table, the in-daemon Axum listener for MVP and the Go-backend Hub for production, the `forward_invoke` dispatch path, the bounded operational receipt ring buffer.

The v0.5.1 backup file is canonical for those details: `AXON-RFC-006-B-easynet-webapp.v0.5.1.tex.bak` in `docs/rfc/`. Where v0.5.1 contradicts v0.6’s invariants (it does not, in any place this author has identified, but future readers should verify), v0.6 wins.

8.2 The MVP: a witness, not a deliverable

[normative] The minimum existence proof of v0.6 is a folder with two files (`hello-world.html`, `style.css`), served through an in-daemon Axum listener bound to `127.0.0.1:8787`, fetched by a real browser, and verified via a kernel-traced path-traversal attempt that returns 404 without the daemon ever issuing a syscall against the target file outside the sandbox.

The MVP is not a product. It is the simplest test that discharges all three invariants simultaneously: Adapter Purity (the Hub forwards to the project’s ability and emits no canonical receipt), Resource Identity (the project mints a v0 identity that admission accepts as a resource subject), and Deterministic Projection (the bytes a browser receives equal the bytes the daemon reads from the published folder, sha256 verified).

8.3 Test stratification

[normative] Three matrices, each layered three deep:

- **Matrix A (unit/integ, in-process):** 13 cases covering each transition’s success and failure paths, each invariant’s enforcement, and concurrent fetches / file-size cap edge cases.
- **Matrix B (CLI surface, out-of-process):** 10 cases covering the `easynet pages {create,list,show,d}` subcommand surface, JSON vs human output, exit codes.
- **Matrix C (Docker e2e, real container, real syscall trace):** 10 cases including a `strace` sidecar that asserts the kernel never reads files outside the published folder during

a path-traversal attack.

A phase ships only when every case in every matrix passes at all three layers (process / wire / state). The matrices are specified in detail in the implementation plan (`vectorized-cooking-sparkle.md`) and not duplicated here.

9 Beyond the reference system

9.1 What this RFC unlocks (without specifying)

[derived] With Pages established as the first reference realisation of the Resource Execution Model, four immediate extensions become routine:

1. **Files** (RFC-006-C, projected). A `file` resource kind, with the same `publish` / `fetch` / `unpublish` triple, but no HTTP adapter — the `fetch` surface is direct ability invocation. INV-1 still holds vacuously (no adapter); INV-2 + INV-3 transfer unchanged.
2. **Conversations** (RFC-006-D, projected). A `conversation` resource kind whose `fetch` surface returns a deterministic projection of the conversation state at a specific receipt id (immutable snapshot semantics); state transitions are message appends.
3. **Mission outputs** (RFC-006-E, projected). A `mission_output` resource kind whose state is the JSON of the mission’s terminal step receipt; `fetch` returns the JSON. Aliasing (mutable name pointing at a specific receipt) is the natural “latest run” shape.
4. **Agent-built API surfaces** (RFC-006-F, projected). A `web_app` resource kind that composes `page` (static bytes) with non-static abilities (`<user>.<project>.api.<verb>`). INV-1’s purity confines the static side; the API side declares its own non-determinism (INV-3 does not bind it).

Each of the above is purely additive: they instantiate the Model without changing it. The paradigm scales horizontally through reference systems, not vertically through ad-hoc extensions.

9.2 What this RFC deliberately leaves to future work

[deferred]

- **INV-2 v1 enforcement** (content-addressed project ids). v0.6 ships under the v0 transitional form; v0.7 (or RFC-006-B-Phase-3 in the implementation plan) flips the requirement.
- **Visibility = PRIVATE / SCOPED**. Capability-URL semantics, full-hash project ids, admission policies for SCOPED. The Resource Execution Model has room for these but does not specify them.
- **HTTP/2 / HTTP/3 / WebSocket / SSE semantics under the same paradigm**. Adapter Purity should extend mechanically, but the receipt-tier classification of long-lived connections is non-trivial; future work.
- **The Resource Execution Model as a standalone specification**. v0.6 introduces the Model in service of Pages; v0.7 promotes it to an independent RFC that this appendix references.

10 Coverage check

Rule	Discharged by	Where
URA v4.1.5 §A.URA-7 (3-segment URA)	resource/ability URAs each parse to three segments	§7
URA v4.1.5 §9 (callee $\in$ hub/device/agent)	callee on fetch is agent / <host> ; on serve is hub/01HUB	§5, §7
ONTOLOGY §3.2 (single owner per ability)	ability owner {user, agent, hub, resource} per A-4	§3
AXIOM (every invocation produces a receipt)	two-tier classification (canonical / operational)	§6
INV-1 (Adapter Purity)	Hub serve forwards via standard forward_invoke ; emits no canonical receipt	§4
INV-2 (Resource Identity)	v0 transitional / v1 content-addressed; transition recorded in publish receipt	§4
INV-3 (Deterministic Projection)	fetch receipt binds $\langle R, p, \text{sha256}(\text{bytes}) \rangle$	§4

Closing. HTTP becomes a transport. A folder becomes a resource. Reading bytes becomes a read-only state transition whose receipt witnesses the deterministic projection. Pages is the first instance of a paradigm that does not need a second instance to be valid, but expects one — and another, and another — because the paradigm is the point and the instance is the proof.

The Resource Execution Model is not a feature. It is the discipline under which features become trustworthy. Pages is the first feature for which we wrote the discipline first.